

Welcome to the IS Project. This is the first assignment sheet and it is about the fundamentals of a web search engine. Your task is to implement an indexer that gets HTML documents and stores them in a database, to write a crawler that feeds the indexer while traversing the web given starting URLs, and to implement scoring models and a simple SQL-based query processor that is finally used to execute Google-style keyword (aka. term/words) searches via a command line interface.

- In the kickoff meeting that you attended, we have outlined the general ways to approach the assignments. Most of the assignments have “just” an overridden goal, but are elsewhere, in details, not overly specified. Feel free to realize it in “your” way.
- We limit the usage of available tools and libraries, so use only the standard Java library as made available with release 19 and later. If there are additional libraries/tools you might use, we mention that in the **hints box** after each assignment.
- There are also **question boxes** at some places in the assignment sheets. These explicitly mention questions you should consider before coming to the review meeting (aka. oral exam). These questions are only a subset of possible questions related to the tasks of an assignment, we might ask.
- Again, note that you should generally be able to answer questions about the way you implemented your solution. All members of the group need to show equal participation. So please discuss your solutions and not just work separately on the individual parts of the assignment sheet.
- Plan and implement your code in an extendable manner. You will need to implement additional functionalities of your search engine in the upcoming sheets.

1 Indexer and Crawler

1.1 Implementing an Indexer

In this assignment your task is to implement an indexer such that:

- It parses the text contained in the HTML document, including the application of a stemmer
- “Stopwords” and HTML markup are eliminated
- Available meta data of the HTML page are harnessed
- The outgoing HTML links are saved
- Store the parsed information and links in a Postgres database using JDBC

Implement the creation of the database schema using JDBC, i.e., the DDL statements should be executed using your Java code. Your database schema initially should have the following tables:

```
features(docid, term, term_frequency)
documents(docid, url, crawled_on_date)
links(from_docid, to_docid)
```

The table **features** stores for each document and term contained in it the number of times this term occurs. The table **documents** is for the documents and corresponding meta information and the table **links** is for the link structure.

Hints

- Feel free to extend the above schema with meta information that you encounter for the web pages, links, etc.
- You can obtain the implementation of a stemmer from <http://tartarus.org/martin/PorterStemmer/java.txt>
- A list of stopwords you find here: <http://snowball.tartarus.org/algorithms/english/stop.txt>
- You don't have to worry about the incorrect stemming of German words and distinguishing English from German websites for now. This will be a task in the upcoming assignments. Feel free to prepare your implementation for this, but at this point it's not mandatory.

1.2 Crawler

Write an HTML crawler that starts from a given set of URLs, indexes the pages using the indexer above, and continues crawling pages that are linked to in a breadth-first manner.

- Parameterize the crawler implementation with
 - maximum depth to crawl,
 - maximum number of documents to crawl,
 - boolean flag indicating if the crawler is allowed to leave domain(s) (e.g., cs.uni-kl.de) or has to stay on it,
 - further parameters you think to be useful.
- Keep the crawler state (queue, visited pages, etc.) also in the database in order to stop/restart it at any time.
- Use multiple Java threads to fully exploit the available hardware resources and bandwidth. For multi-threading use transactions with the appropriate database isolation level.
- Be “nice”, avoid many accesses within a short amount of time to the same domain by limiting the crawling speed.

Hints

- Be aware of cycles.
- You can use a library like JTidy (<http://jtidy.sourceforge.net> or the latest version supporting HTML5 <https://github.com/jtidy/jtidy>) for cleaning up the HTML in order to convert it to XHTML. This is needed if you want to get the outgoing links with XPATH. You can also get the links by manually parsing the <a> tags. However, **you need to implement your own HTML parser for the content of the webpage.**
- Using the following URL-normalization library is allowed: <https://github.com/shekhargulati/urllcleaner>. However, you can only use the class **UrlCleaner** and not the class UrlExtractor.

2 TF*IDF Score Computation

Now as the database contains documents and their features and links, a way to assess the suitability of a document with respect to a given term, respectively multiple terms, needs to be realized. For now, the link structure is ignored. The task is to implement a simple TF*IDF scoring model, a combination of a *term frequency* (*tf*) measure and a measure for the *inverse document frequency* (*idf*). If $f(t, d)$ denotes the absolute frequency of how often term t occurs in document d , then a possible *tf* measure is calculated as

$$tf(t, d) = 1 + \log f(t, d)$$

or zero if $f(t, d)$ is zero. For the inverse document frequency, we count for each term in how many documents it appears. It is used to dampen the influence of terms that appear extremely often and to push the importance of terms that are rather rare. Hence, if $|\{d \in D : t \in d\}|$ denotes for a collection D of documents the number of documents that contain term t , the *idf* measure is computed as

$$idf(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

where N is the total number of documents.

- Implement SQL queries that compute the term frequency, the inverse document frequency, and the combined score and store this information in the database.
- For that you need to extend the DB schema appropriately.
- If the crawler is re-started on an already existing DB, note that the TF*IDF scores need to be recomputed.

Hints

- The Wikipedia page of the TF*IDF measure <http://en.wikipedia.org/wiki/Tf-idf> is very decent and also contains an example.

Questions

- Where did you add the additional columns to accommodate the above frequency counts and scores, and why?
- Why could it be useful to introduce a bit of redundancy in the DB schema/data?
- Why are, in general, additionally added documents requiring the scores (TF*IDF) to be recomputed?

3 Query Processing and Command Line Interface

3.1 SQL Queries

Consider a Google-style keyword query of m terms $t_1, t_2, \dots, t_m$.

Your task is to design SQL queries that compute Google-style queries over this database by using the **features** table of your DB schema (including the added TF*IDF score column).

```
SELECT docid, ... as score
FROM features WHERE ...
```

The result of the devised query are the top- k , for a given parameter k (e.g., 5), of documents that *match* the terms $t_1, t_2, \dots, t_m$ specified in the query. This result list is ordered by the aggregated score of each document, in descending order (i.e., best ones first).

Realize the following two modes:

1. **Conjunctive:** Documents are considered matching a query if they contain *all* of the query's terms.
2. **Disjunctive:** Documents are considered matching a query if they contain *any* of the query's terms.

3.2 JDBC and Command Line Interface

Now, we also need an user interface in order to administer and test the implemented functionalities of the system, e.g., executing the SQL queries against the database.

- Implement a Java Class/Method that takes as input a keyword query (i.e., list of query terms, result size k , indicator of disjunctive vs. conjunctive mode) instantiates one of the above SQL queries according to the input, and executes it using JDBC.
- Implement a Command Line Interface (CLI) that connects at startup to a specified database instance (crawl) from the console window (i.e., terminal) and accepts blank-separated keywords as queries. The top-5 results (less if less documents qualify) are printed to the standard output, including rank (1, 2, ... etc.), url, and achieved score.

3.3 Indices

The performance (runtime) of the above queries can be influenced by the creation of appropriate indices. Add their creation to the above database indexing steps.

Questions

- What kind of indices are required?
- Why are the indices that you have created required?
- When is a good time to create these indices?

4 User and JSON Interface

4.1 User Interface

Design and implement an HTML user interface using the Apache Tomcat framework.

- Similar to the Google interface, the interface should consist of a input form and a submit button (and maybe/optionally a cool logo).

- The users can enter keyword queries that are processed in your engine and results are accordingly shown, in ranked form. For simplicity, limit the number of displayed results to 20.
- Implement the **site:** and **quotation marks** (“”) operators in your search. The **site:** operator retrieves results only from certain sites or domains. A query containing a keyword put in **quotation marks** requires the keyword to be present in every result returned. If there are multiple such keywords in a single query, then each keyword needs to be put in quotation marks separately.
- Any other functionality like the “I am feeling lucky” button etc. are not required but feel free to implement some of them and/or polish the appearance of your interface.

4.2 JSON Interface

Expose your implementation to other machines. For this, implement a simple Java Servlet that is accepting query and maximum number of requested query results, k , via an HTTP GET request. The default query mode should be disjunctive—using the quotation marks operator, of course, some or all keywords might be required. The Java Servlet should return the results in a JSON format following the example below:

```
{
  "resultList": [
    {
      "rank": 1,
      "url": "http://cs.uni-kl.de/en/organisation/",
      "score": 0.9
    },
    {
      "rank": 2,
      "url": "http://cs.uni-kl.de/en/studium/",
      "score": 0.7
    },
    {
      "rank": 3,
      "url": "www.cs.uni-kl.de",
      "score": 0.65
    },
    ...
  ],
  "query": {
    "k": 5,
    "query": "computer_science_department"
  },
  "stat": [
    {
      "term": "computer",
      "df": 50
    },
    ...
  ],
  "cw": 10000
}
```

The df attribute is the number of documents in the collection containing the specific term, while cw is the number of terms in the collection. Note that you will need these attributes in the next assignments.